P2752R3
Static storage for braced initializers

Published Proposal, 2023-06-14

Author:
Arthur O'Dwyer

Audience:
CWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:
15

Current Source:
github.com/Quuxplusone/draft/blob/gh-pages/d2752-static-storage-for-braced-initializers.bs

Current:
rawgit.com/Quuxplusone/draft/gh-pages/d2752-static-storage-for-braced-initializers.html

---

Abstract

Initializing a vector from a *braced-initializer-list* like `{1,2,3}` copies the data from static storage to a backing array on the stack, and thence into the vector. This wastes CPU cycles, and it wastes stack space. Eliminate the waste by letting a `std::initializer_list<T>`'s backing array occupy static storage.

Table of Contents

1 Changelog

2 Background

2.1 Workarounds

3 Solution

4 Mutable members

5 Constexpr evaluation

5.1 However...

6 Implementation experience

7 Proposed wording relative to the current C++23 draft

7.1 Addition to Annex C

8 Acknowledgments

References

Informative References

§ 1. Changelog

- R3 (mid-Varna 2023):
  - Add that this proposal is already DR'ed in GCC trunk.
  - EWG polled "Forward to CWG for C++26 with the addition of an Annex C entry and discussion of no-intent to change storage duration," 6–23–1–0–0.
  - After EWG: Add Annex C entry. Add the words "and with the understanding that `__b` does not outlive the call to `f`" in Example 12.
  - After CWG: Add discussion of `mutable` members, and Example 12's subexample `r`. Add the constexpr § 5.1 However... section. Headed to the straw-polls page.
- R2 (pre-Varna 2023):
  - Add discussion of unspecified behavior in constexpr evaluation.
- R1 (post-Issaquah 2023):
  - Remove discussion of mangling; it's not a problem. Add discussion of GCC's `-fmerge-all-constants`.
  - Proposed wording reviewed by Jens Maurer.

§ 2. Background

[dcl.init.list]/5–6 says:

> An object of type `std::initializer_list<E>` is constructed from an initializer list as if the implementation generated and materialized a prvalue of type "array of *N* `const E`", where *N* is the number of elements in the initializer list. Each element of that array is copy-initialized with the corresponding element of the initializer list, and the `std::initializer_list<E>` object is constructed to refer to that array.
>
> [*Example 12:*
>
> ```
> struct X {
>   X(std::initializer_list<double> v);
> };
> X x{ 1,2,3 };
> ```
>
> The initialization will be implemented in a way roughly equivalent to this:
>
> ```
> const double __a[3] = {double{1}, double{2}, double{3}};
> X x(std::initializer_list<double>(__a, __a+3));
> ```
>
> assuming that the implementation can construct an `initializer_list` object with a pair of pointers. *—end example*] [...]
>
> **[*Note 6: The implementation is free to allocate the array in read-only memory if an explicit array with the same initializer can be so allocated. —end note*]**

In December 2022, Jason Merrill observed ([CoreReflector]) that this note isn't saying much. Consider the following translation unit:

```
void f(std::initializer_list<int> il);

void g() {
    f({1,2,3});
}

int main() { g(); }
```

Can the backing array for `{1,2,3}` be allocated in static storage? No, because `f`'s implementation might look like this:

```
void g();

void f(std::initializer_list<int> il) {
    static const int *ptr = nullptr;
    if (ptr == nullptr) {
        ptr = il.begin();
        g();
    } else {
        assert(ptr != il.begin());
    }
}
```

A conforming C++23 implementation must compile this code in such a way that the two temporary backing arrays — the one pointed to by `il.begin()` during the first recursive call to `f`, and the one pointed to by `il.begin()` during the second recursive call to `f` — have distinct addresses, because that would also be true of "an explicit array [variable]" in the scope of `g`.

All three of GCC, Clang, and MSVC compile this code in a conforming manner. (Godbolt.)

Expert programmers tend to understand that when they write

```
std::vector<int> v = {1,2,3,4,5,6,7,8,9,10};
```

they're getting a copy of the data from its original storage into the heap-allocated STL container: obviously the data has to get from point A to point B somehow. But even expert programmers are surprised to learn that there are actually *two* copies happening here — one from static storage onto the stack, and another from stack to heap!

Worse: [P1967R9] (adopted for C++26) allows programmers to write

```
std::vector<char> v = {
    #embed "2mb-image.png"
};
```

Suppose "2mb-image.png" contains 2MB of data. Then the function that initializes `v` here will create a temporary backing array of size 2MB. That is, we're adding 2MB to the stack frame of that function. Suddenly your function's stack frame is 2MB larger than you expected! This applies even if the

"function" in question is the compiler-generated initialization routine for a dynamically initialized global variable `v`. You might not even control the function whose stack frame is blowing up.

This kind of thing was always possible pre-P1967, but was hard to hit in human-generated code, because braced initializer lists were generally short. But post-P1967, this is easy to hit. I think this stack-frame blowup will become a well-known problem unless we solve it first.

§ 2.1. Workarounds

This code creates a 2MB backing array on the stack frame of the function that initializes `v`:

```
std::vector<char> v = {
    #embed "2mb-image.png"
};
```

This code does not:

```
static const char backing[] = {
    #embed "2mb-image.png"
};
std::vector<char> v = std::vector<char>(backing, std::end(backing));
```

So the latter is a workaround. But it shouldn't be necessary to work around this issue; we should fix it instead.

JeanHeyd Meneide points out that C's "compound literals" have the same problem, and they chose to permit the programmer to work around it in C23 by adding storage class annotations directly to their literals, like this:

```
int f(const int *p);
int x = f((static const int[]){ // C23 syntax
    #embed "2mb-image.png"
});
```

Zhihao Yuan's [P2174R1] proposes adding C-compatible compound literals to C++, but does not propose storage-class annotations. Anyway, that's not a solution to the `std::initializer_list` problem, because `initializer_list` syntax is lightweight by design. We don't need new syntax; we just need the existing syntax to Do The Right Thing by default.

§ 3. Solution

Essentially we want the semantics of braced initializer lists to match the semantics of string literals ([lex.string]/9).

We want to encourage tomorrow's compilers to avoid taking up any stack space for constant-initialized initializer lists. If possible I'd like to *mandate* they not take any stack space, but I don't currently know how to specify that. Quality implementations will take advantage of this permission anyway.

```
std::initializer_list<int> i1 = {
    #embed "very-large-file.png"  // OK
};

void f2(std::initializer_list<int> ia,
        std::initializer_list<int> ib) {
    PERMIT(ia.begin() == ib.begin());
}
int main() {
    f2({1,2,3}, {1,2,3});
}
```

We want to permit tomorrow's compilers to share backing arrays with elements in common, just like today's compilers can share string literals. High-quality implementations might take advantage of this permission; mainstream compilers probably won't bother.

```
const char *p1 = "hello world";
const char *p2 = "world";
PERMIT(p2 == p1 + 6);  // today's behavior

std::initializer_list<int> i1 = {1,2,3,4,5};
std::initializer_list<int> i2 = {2,3,4};
PERMIT(i1.begin() == i2.begin() + 1);  // tomorrow's proposed behavior
```

We even intend to permit tomorrow's compilers to share backing arrays between static and dynamic initializer lists, if they can prove it's safe.

The lifetime of the backing array remains tied to its original `initializer_list`. Accessing the backing array outside its lifetime (as in `f4`) remains UB. [class.base.init]/11, which makes it ill-

formed to bind a reference member to a temporary (as in `C5`), is also unaffected. (As of December 2022, Clang diagnoses `C5`; GCC doesn't; MSVC gives a possibly unrelated error message.)

```
const int *f4(std::initializer_list<int> i4) {
    return i4.begin();
}
int main() {
    const int *p = f4({1,2,3});
    std::cout << *p;  // still UB, not OK
}

struct C5 {
    C5() : i5 {1,2,3} {}  // still ill-formed, not OK
    std::initializer_list<int> i5;
};
```

We do not permit deferring, combining, or omitting the side effects of constructors or destructors related to the backing array. In practice, compilers will "static-fy" backing arrays of types that are constinit-constructible and trivially destructible, and not "static-fy" anything else.

```
struct C6 {
    constexpr C6(int i) {}
    ~C6() { printf(" X"); }
};
void f6(std::initializer_list<C6>) {}
int main() {
    f6({1,2,3}); // must still print X X X
    f6({1,2,3}); // must still print X X X
}
```

§ 4. Mutable members

> NOTE: This section is new in R3. Thanks to Hubert Tong and CWG for the tip.

Types with `mutable` data members can prevent the optimization from occurring. That is, vendors must make the following example succeed and not throw.

```
struct S {
  constexpr S(int i) : i(i) {}
  mutable int i;
};

void f(std::initializer_list<S> il) {
  if (il.begin()->i != 1) throw;
  il.begin()->i = 4;
}

int main() {
  for (int i = 0; i < 2; ++i) {
    f({1,2,3});
  }
}
```

The first call to `f` receives an `il` backed by `{1,2,3}`, and modifies it to `{4,2,3}`. The second call to `f` must receive an `il` again backed by `{1,2,3}`. The implementation is not permitted to give the second call an `il` backed by `{4,2,3}`; that would simply be a wrong-codegen bug in the implementation.

(GCC's initial implementation had this bug, but a fix is in the works.)

Vendors are expected to deal with this by simply disabling their promote-to-shared-storage optimization when the element type (recursively) contains any mutable bits.

§ 5. Constexpr evaluation

> NOTE: This section is new in R2 and updated in R3.

Lénárd Szolnoki points out that P2752 introduces a new way for a pointer comparison to be "unspecified." This is fine, but it is also observable.

```
constexpr bool f9(const int *p) {
    std::initializer_list<int> il = {1,2,3};
    return p ? (p == il.begin()) : f9(il.begin());
```

```
}

inline constexpr bool b9 = f9(nullptr);
```

Here the compile-time value of `b9` depends on whether `{1,2,3}`'s backing array is on the stack or not. Today, this program is well-formed and `b9` is `false`. *According to one vendor's interpretation*, P2752 makes this program ill-formed, because we make the result of `p == il.begin()` unspecified, and therefore ([expr.const]/5.24) not a core constant expression.

Remove the `constexpr` from that program, and it becomes well-formed, but the programmer might notice that the initialization of inline variable `b9` is no longer being done statically; instead it's initialized dynamically, so that we can ensure that every TU gets the same value for it.

Here's one more example:

```
template<class T>
constexpr bool f10(T s, T t) {
    return s.begin() == t.begin();
}

constexpr bool b10a = f10<std::string_view>("abc", "abc");
    // Today: Ill-formed because of unspecified comparison
    // Clang rejects; GCC, MSVC, EDG say true

constexpr bool b10b = f10<std::string_view>("abc", "def");
    // Today: Well-formed false
    // Clang rejects; GCC, MSVC, EDG say false

constexpr bool b10c = f10<std::initializer_list<int>>({1,2,3}, {1,2,3});
    // Today: Well-formed false (all vendors agree)
    // Tomorrow: Ill-formed because of unspecified comparison
    //   (but expect vendor divergence as above)

constexpr bool b10d = f10<std::initializer_list<int>>({1,2,3}, {4,5,6});
    // Today: Well-formed false (all vendors agree)
    // Tomorrow: Well-formed false
    //   (but expect vendor divergence as above)
```

§ 5.1. However...

CWG discussion revealed complexity here. Right now, empirically, Clang interprets [expr.const]/5.24 as meaning that any pointer comparison whose result *happens to be* unspecified necessarily is not a core constant. The GCC implementor in the room initially felt the same way (i.e. that the above behavior was a GCC bug). But another CWG participant put forth the interpretation that [expr.const] intended to classify only a very narrow subset of expressions as "[pointer comparisons] where the result is unspecified," i.e., those covered specifically by [expr.eq]/3.1. Under that interpretation, "unspecified-ness" is a point property, not something that needs to be tracked by dataflow analysis during constexpr evaluation.

```
constexpr const char *a = "abc";
constexpr const char *b = "abc";
constexpr bool f11() { return a == b; }
static_assert(f11() || !f11());
    // Clang rejects; GCC, MSVC, EDG accept
```

Here it is unspecified whether `a` and `b` share storage, so `a == b`'s result is unspecified (either true or false). Clang diagnoses `f11` as ill-formed, because it claims that `a == b` is a pointer comparison with an unspecified result ([expr.const]/5.24).

According to the non-Clang interpretation, the constexpr-evaluator is supposed to decide early on whether `a` and `b` share storage or not. The result of that decision is unspecified. But after that point, comparisons such as `a == b` are well-defined true or well-defined false; the only kind of "equality operator where the result is unspecified" according to [expr.eq]/3.1 (and thus not a core constant expression by [expr.const]/5.24) would be a one-past-the-end comparison. We're not doing that here.

Clang's interpretation leads to a reductio ad absurdam:

```
int a[10];
constexpr int inc(int& i) { return (i += 1); }
constexpr int twox(int& i) { return (i *= 2); }
constexpr int f(int i) { return inc(i) + twox(i); }
constexpr bool g() { return &a[f(1)] == &a[6]; }
constexpr bool b11 = g();
    // Today: "Ill-formed" according to Clang's interpretation
    //   But Clang, GCC, MSVC, EDG all report "true"
```

Here `f(1)`'s result is unspecified (either 6 or 8 depending on order of evaluation), so the pointer equality comparison `&a[f(1)] == &a[6]` produces an unspecified answer (either true or false

depending on `f(1)`); and yet, not even Clang diagnoses the use of `g()` as a core constant expression. That is, Clang's draconian interpretation of [expr.const]/5.24 cannot be sustained even by Clang itself.

As a result of this discussion, CWG decided to strike the part of the Annex C entry dealing with constexpr evaluation. The Annex C entry now mentions only that some well-defined results have become unspecified, and doesn't imply that anything might become ill-formed.

> NOTE: § 5.1 However... was written after the CWG discussion, but the proposed wording below is exactly as CWG approved it; those Annex C diffs were made live in the CWG meeting and approved as they were made.

§ 6. Implementation experience

I have an experimental patch against Clang trunk; see [Patch]. It is incomplete, and (as of this writing) buggy; it has not received attention from anyone but myself. You can experiment with it on Godbolt Compiler Explorer; just use the P1144 branch of Clang, with the `-fstatic-init-lists` command-line switch.

In June 2023, Jason Merrill implemented this proposal in GCC trunk (the rising GCC 14), as a DR in all language modes. Compare the behavior of GCC 13 against GCC trunk.

§ 7. Proposed wording relative to the current C++23 draft

Modify [dcl.init.list]/5–6 as follows:

An object of type `std::initializer_list<E>` is constructed from an initializer list as if the implementation generated and materialized a prvalue of type "array of *N* `const E`", where *N* is the number of elements in the initializer list; this is called the initializer list's *backing array*. Each element of ~~that array~~ the backing array is copy-initialized with the corresponding element of the initializer list, and the `std::initializer_list<E>` object is constructed to refer to that array.

~~[*Example 12:*~~

```
struct X {
  X(std::initializer_list<double> v);
};
X x{ 1,2,3 };
```

~~The initialization will be implemented in a way roughly equivalent to this:~~

```
const double __a[3] = {double{1}, double{2}, double{3}};
X x(std::initializer_list<double>(__a, __a+3));
```

~~assuming that the implementation can construct an `initializer_list` object with a pair of pointers. —*end example*]~~

Whether all backing arrays are distinct (that is, are stored in non-overlapping objects) is unspecified.

The backing array has the same lifetime as any other temporary object, except that initializing an `initializer_list` object from the array extends the lifetime of the array exactly like binding a reference to a temporary.

[*Example 12:*

```
void f(std::initializer_list<double> il);
void g(float x) {
  f({1, x, 3});
}
void h() {
  f({1, 2, 3});
}

struct A {
  mutable int i;
};
void q(std::initializer_list<A>);
void r() {
  q({A{1}, A{2}, A{3}});
}
```

The initializations can be implemented in a way roughly equivalent to this:

```
void g(float x) {
  const double __a[3] = {double{1}, double{x}, double{3}}; // backing array
  f(std::initializer_list<double>(__a, __a+3));
}
void h() {
  static constexpr double __b[3] = {double{1}, double{2}, double{3}}; // backing array
  f(std::initializer_list<double>(__b, __b+3));
}
void r() {
  const A __c[3] = {A{1}, A{2}, A{3}}; // backing array
  q(std::initializer_list<A>(__c, __c+3));
}
```

assuming that the implementation can construct an `initializer_list` object with a pair of pointers, and with the understanding that `__b` does not outlive the call to `f`. *—end example*]

[*Example 13:*

```
typedef std::complex<double> cmplx;
std::vector<cmplx> v1 = { 1, 2, 3 };
void f() {
  std::vector<cmplx> v2{ 1, 2, 3 };
  std::initializer_list<int> i3 = { 1, 2, 3 };
}
struct A {
  std::initializer_list<int> i4;
  A() : i4{ 1, 2, 3 } {}            // ill-formed, would create a dangling reference
};
```

For `v1` and `v2`, the `initializer_list` object is a parameter in a function call, so the array created for `{ 1, 2, 3 }` has full-expression lifetime. For `i3`, the `initializer_list` object is a variable, so the array persists for the lifetime of the variable. For `i4`, the `initializer_list` object is initialized in the constructor's *ctor-initializer* as if by binding a temporary array to a reference member, so the program is ill-formed. *—end example*]

[*Note 6: The implementation is free to allocate the array in read-only memory if an explicit array with the same initializer can be so allocated. —end note*]

§ 7.1. Addition to Annex C

Modify Annex C [diff.cpp20.expr] as follows:

Affected subclause: [dcl.init.list]

Change: Pointer comparisons between `initializer_list` objects' backing arrays are unspecified.

Rationale: Permit the implementation to store backing arrays in static read-only memory.

Effect on original feature: Valid C++ 2023 code that relies on the result of pointer comparison between backing arrays may change behavior. For example:

```
bool ne(std::initializer_list<int> a, std::initializer_list<int> b) {
  return a.begin() != b.begin() + 1;
}
bool b = ne({2,3}, {1,2,3}); // unspecified result; previously false
```

§ 8. Acknowledgments

- Thanks to Jason Merrill for the original issue, and to Andrew Tomazos for recommending Arthur write this paper.
- Thanks to Jens Maurer for reviewing P2752R1's proposed wording.
- Thanks to Lénárd Szolnoki for pointing out the ramifications for constexpr.

§ References

§ Informative References

[CoreReflector]
Jason Merrill. *[isocpp-core] initializer_list and read-only memory*. December 2022. URL: https://lists.isocpp.org/core/2022/12/13625.php

[P1967R9]
JeanHeyd Meneide. *#embed - a scannable, tooling-friendly binary resource inclusion mechanism*. October 2022. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1967r9.html

[P2174R1]
Zhihao Yuan. *Compound Literals*. April 2022. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2174r1.html

[Patch]
Arthur O'Dwyer. *Implement P2752R0 Static storage for braced initializers*. January 2023. URL: https://github.com/Quuxplusone/llvm-project/tree/p2752